

Agenten-Anleitung für die Website-Prüfung

Diese Anleitung erklärt, welche lokalen Agenten und Skills im Projekt vorhanden sind, wofür sie zuständig sind und in welcher Reihenfolge sie eingesetzt werden sollten. Ziel ist ein kontrollierter Workflow bis zu einem professionellen Launch.

Grundregel: Spezialagenten prüfen read-only und berichten. Code-Änderungen werden erst nach ausdrücklicher Freigabe durch den Hauptagenten vorbereitet.

1. Arbeitsprinzip

Arbeite in drei Stufen:

- 1. **Vor dem Bauen:** prüfen, was im Projekt bereits existiert.
- 2. **Nach Änderungen:** Regeln, Accessibility und technische Checks ausführen.
- 3. **Vor Launch:** visuelle, inhaltliche, SEO-, Performance- und Gesamtprüfung machen.

Nicht jeder Agent muss bei jeder Aufgabe laufen. Nutze gezielt den Agenten, der zum Risiko der aktuellen Änderung passt.

2. Bestehende Pflichtagenten

Agent	Zuständig für	Wann einsetzen
codebase_explorer	Findet vorhandene Komponenten, Utilities und Types.	Vor neuen Komponenten, größeren Refactorings oder wenn unklar ist, ob es schon etwas Passendes gibt.
quality_reviewer	Prüft AGENTS.md-Regeln: Kommentare, Verbote, Naming und Definition of Done.	Nach jedem schreibenden Task und vor jeder Fertigmeldung.
a11y_checker	Prüft Accessibility: Semantik, Überschriften, Formulare, Tastatur, Bilder, Fokus.	Bei jeder Änderung an UI, Formularen oder interaktiven Elementen.

3. Neue Audit-Agenten

Agent	Zuständig für	Typischer Einsatz
typography_checker	Schriftgrößen, Fluid-Type-Klassen, Lesebreiten, Zeilenumbrüche, Button-Wrapping.	Nach Text-, Layout- oder Hero/Card-Änderungen.
mobile_visual_checker	Mobile Screenshots, Overflow, CTA-Erreichbarkeit, Header/Nav, Touch-Ziele.	Nach größeren UI-Änderungen und vor Launch.
desktop_visual_checker	Desktop-Screenshots, Weißraum, Bildbeschnitt, visuelle Hierarchie, Premium-Eindruck.	Nach Layoutänderungen und am Ende einer Designrunde.
conversion_reviewer	Telefon, WhatsApp, Kontaktformular, CTA-Struktur, Weg zur Anfrage.	Bei Leistungsseiten, Kontaktbereichen und zwingend vor Launch.
local_seo_reviewer	Berlin-Bezug, Titles, Descriptions, Leistungsseiten, interne Verlinkung, strukturierte Daten.	Nach Inhaltsänderungen und vor Launch.
legal_trust_reviewer	Impressum, Datenschutz, Firmenangaben, Kontaktklarheit, Trust-Risiken.	Vor Launch und bei Footer/Kontakt/Datenschutz-Änderungen.
performance_budget_reviewer	Bilder, Fonts, JS/CSS-Größe, Client Components, Build-Ausgaben, Core-Web-Vitals-Risiken.	Nach dem Build, nach neuen Bildern/Animationen und vor Launch.
content_consistency_reviewer	Tonalität, gleiche Leistungsnamen, widersprüchliche Aussagen, Platzhalter, Mehrsprachigkeitsrisiken.	Nach Textänderungen und vor Launch.
premium_reviewer	Hochwertiger Gesamteindruck, Vertrauen, Bildsprache, Professionalität.	Am Ende einer Design-/Content-Runde.
launch_reviewer	Finale Zusammenführung aller Findings mit P0/P1/P2/P3-Prioritäten.	Ganz am Ende, nachdem Spezialprüfungen gelaufen sind.

4. Skills

Skill	Zuständig für	Wann einsetzen
<code>next-router-check</code>	Prüft App-Router-Routen auf <code>loading.tsx</code> , <code>error.tsx</code> und <code>not-found.tsx</code> .	Wenn neue oder geänderte Routen unter <code>/app</code> betroffen sind.
<code>site-launch-audit</code>	Koordiniert die komplette Launch-Prüfung mit Spezialagenten, Screenshots und Abschlussreview.	Wenn die ganze Website strukturiert vor Launch geprüft werden soll.

5. Empfohlene Reihenfolgen

Neue Komponente bauen

1. `codebase_explorer`
2. Umsetzung durch den Hauptagenten
3. `a11y_checker`, wenn UI betroffen ist
4. `quality_reviewer`
5. `npm run lint`, `npx tsc --noEmit`, `npm run build`

Texte ändern

1. `content_consistency_reviewer`
2. `typography_checker`
3. `conversion_reviewer`, wenn CTAs oder Kontaktwege betroffen sind
4. `quality_reviewer`

Layout oder Design ändern

1. `codebase_explorer`, wenn neue Komponenten möglich sind
2. Umsetzung durch den Hauptagenten
3. `typography_checker`
4. `mobile_visual_checker`
5. `desktop_visual_checker`
6. `a11y_checker`
7. `quality_reviewer`

Route oder neue Seite ändern

1. `codebase_explorer`
2. Umsetzung durch den Hauptagenten
3. `next-router-check`
4. `a11y_checker`
5. `quality_reviewer`
6. `local_seo_reviewer`
7. Technische Checks ausführen

Komplette Launch-Prüfung

1. `npm run lint`
2. `npx tsc --noEmit`
3. `npm run build`
4. `typography_checker`
5. `mobile_visual_checker`
6. `desktop_visual_checker`
7. `conversion_reviewer`
8. `local_seo_reviewer`
9. `legal_trust_reviewer`
10. `performance_budget_reviewer`
11. `content_consistency_reviewer`
12. `premium_reviewer`
13. `a11y_checker`
14. `quality_reviewer`
15. `launch_reviewer`

6. Beispiel-Prompts

Kompletter Launch-Audit

Nutze `site-launch-audit` und führe einen kompletten Launch-Audit durch. Prüfe zuerst Build, Lint und TypeScript, dann Mobile, Desktop, Typografie, Conversion, SEO, Legal/Trust, Performance, Content, Premium und fasse alles mit `launch_reviewer` in P0/P1/P2/P3 zusammen.

Nur Mobile prüfen

Starte `mobile_visual_checker` für `/`, `/leistungen` und `/kontakt` mit 390x844 und 430x932. Berichte nur konkrete mobile Blocker und Reibungspunkte.

Nur Desktop/Premium prüfen

Starte `desktop_visual_checker` und `premium_reviewer` für alle Hauptseiten. Prüfe Zeilenumbrüche, Weißraum, Bildbeschnitt, visuelle Hierarchie und hochwertigen Gesamteindruck.

Nach Findings gezielt umsetzen

Setze bitte nur die P1-Findings aus dem `launch_reviewer` um. Vorher kurz sagen, welche Dateien du ändern wirst. Keine P2/P3-Punkte anfassen.

7. Prioritäten

Priorität	Bedeutung
P0	Launch unmöglich. Muss sofort gelöst werden.
P1	Vor Launch beheben. Starkes Risiko für Nutzer, SEO, Vertrauen oder Funktion.
P2	Sollte vor Launch verbessert werden, blockiert aber nicht zwingend.
P3	Nach Launch möglich.

8. Wichtig für kontrolliertes Arbeiten

- Spezialagenten ändern keine Dateien.
- Der Hauptagent setzt erst nach deiner Freigabe um.
- Bei neuen Komponenten zuerst `codebase_explorer` nutzen.
- Bei UI immer `a11y_checker` nachziehen.
- Nach schreibenden Tasks immer `quality_reviewer` und die technischen Checks ausführen.
- Vor Launch nicht einzelne Meinungen sammeln, sondern mit `launch_reviewer` priorisiert zusammenführen.

Stand: erstellt für das Projekt Unext-Performance. Diese Anleitung beschreibt die lokalen Agenten und Skills, die im Repository unter `.codex/agents` und `.agents/skills` liegen.